Technical Implementation Report

Implementation of an Observability Solution on Ubuntu with Prometheus, Grafana, and Node Exporter

Confidentiality Notice

This is a publication-adapted version of the original implementation document.

A few environment details, service identifiers, and business references were intentionally removed or generalized. The technical flow, deployment logic, and validation steps were preserved.

Document Classification

Publication-adapted technical implementation report.

1. Executive Summary

This document describes the implementation of a baseline observability solution on Ubuntu using Docker Compose, Prometheus, Grafana, and Node Exporter.

The implementation was designed to provide host-level monitoring, operational dashboards, and foundational alerting capabilities while coexisting with services already running on the target host. The work covered deployment design, port allocation, configuration management, metrics validation, dashboard integration, alert rule definition, and restart continuity considerations.

This report has been adapted for public sharing as an example of enterprise-style technical implementation documentation. Certain references that would normally appear in an internal delivery record have been removed or generalized for confidentiality reasons.

2. Objective and Scope

2.1 Objective

The objective of the implementation was to establish a reusable and operationally sound observability layer on Ubuntu capable of:

- collecting host-level infrastructure metrics
- visualizing operational state through dashboards
- defining threshold-based alerting rules
- validating observability behavior under controlled system load
- preserving service coexistence with pre-existing workloads on the same host
- enabling operational continuity through restart policies

2.2 Scope

The scope of this implementation included:

- deployment of Prometheus for metrics collection and alert evaluation
- deployment of Grafana for dashboard visualization
- deployment of Node Exporter for Linux host metrics exposure
- configuration of Docker Compose for reproducible deployment
- configuration of Prometheus scrape targets
- creation and validation of alerting rules
- dashboard import and data source integration
- basic operational validation through controlled CPU load generation
- verification of restart behavior through Docker restart policies

The scope did not include the following items in this public version:

- business service identification
- customer application naming
- notification routing integrations such as email, Slack, or incident platforms
- high availability design
- retention tuning for enterprise-scale metrics volume
- authentication hardening beyond default implementation setup
- backup or disaster recovery procedures specific to the original environment

3. Background and Implementation Context

The observability solution was deployed on an Ubuntu host that already had other locally running services in operation.

Because the host was not dedicated exclusively to observability, implementation decisions had to account for coexistence with existing workloads, port allocation constraints, and minimal disruption to pre-existing services.

For confidentiality reasons, this document does not disclose the names, business role, or technical ownership of the existing services present on the host at the time of implementation.

3.1 Port Allocation Strategy

The following host ports were assigned to the observability stack:

- Grafana -> `3001`
- Prometheus -> `9090`
- Node Exporter -> `9100`

The Grafana port was intentionally remapped from its default host exposure to avoid conflict with a service already bound to the default port in the target environment.

4. Solution Overview

4.1 Components

Prometheus

Prometheus was implemented as the central metrics collection and alert evaluation component.

Node Exporter

Node Exporter was deployed to expose Linux host metrics, including CPU, memory, disk, uptime, filesystem, load, and network-related information.

Grafana

Grafana was implemented as the visualization layer, consuming metrics from Prometheus through a configured data source and presenting them through imported dashboards.

Docker Compose

Docker Compose was selected as the deployment mechanism in order to provide a simple, repeatable, and isolated method of implementing the solution.

4.2 Logical Flow

The implemented flow is summarized as follows:

1. Node Exporter exposes host metrics over HTTP.
2. Prometheus scrapes those metrics at a fixed interval.
3. Grafana queries Prometheus as a data source.
4. Alert rules are evaluated by Prometheus.
5. Operational validation is performed through controlled load generation and observation of state transitions.

5. Directory and File Structure

The implementation used the following structure:

```
~/observability/
├── docker-compose.yml
├── README.md
└── prometheus/
    ├── prometheus.yml
    └── rules/
        └── alerts.yml
```

5.1 Purpose of Each File

- `docker-compose.yml`: defines the containerized services, ports, mounts, and restart behavior

- `prometheus/prometheus.yml`: defines scrape targets and rule loading
- `prometheus/rules/alerts.yml`: defines alert expressions, thresholds, labels, and annotations
- `README.md`: provides operational and descriptive documentation for the solution

6. Pre-Implementation Verification

Prior to deployment, the host was checked to confirm that the required tooling was available.

6.1 Verify Docker and Docker Compose

```
docker --version
docker compose version
```

If both commands return version information, the host is ready for the deployment procedure.

6.2 Create Working Directories

```
mkdir -p ~/observability/prometheus
cd ~/observability
```

6.3 Confirm Current Working Directory

```
pwd
```

7. Prometheus Configuration

The initial Prometheus configuration file was created at:

```
~/observability/prometheus/prometheus.yml
```

Command:

```
nano ~/observability/prometheus/prometheus.yml
```

Initial configuration:

```yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']
```

```
  - job_name: 'node_exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

7.1 Technical Explanation

- `scrape_interval: 15s` defines the metrics collection interval.
- The `prometheus` job enables self-scraping for Prometheus monitoring.
- The `node_exporter` job enables collection of Linux host metrics.

7.2 Save Procedure in Nano

- `Ctrl + O`
- `Enter`
- `Ctrl + X`

7.3 File Verification

```
cat ~/observability/prometheus/prometheus.yml
```

8. Docker Compose Deployment Definition

The main deployment file was created at:

```
~/observability/docker-compose.yml
```

Command:

```
nano ~/observability/docker-compose.yml
```

Configuration:

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: observability_prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/rules:/etc/prometheus/rules:ro
      - prometheus_data:/prometheus

  grafana:
```

```
    image: grafana/grafana:latest
    container_name: observability_grafana
    restart: unless-stopped
    ports:
      - "3001:3000"
    volumes:
      - grafana_data:/var/lib/grafana

  node-exporter:
    image: prom/node-exporter:latest
    container_name: observability_node_exporter
    restart: unless-stopped
    ports:
      - "9100:9100"
    command:
      - '--path.rootfs=/host'
    pid: host
    volumes:
      - '/:/host:ro,rslave'

volumes:
  prometheus_data:
  grafana_data:
```

8.1 Technical Considerations

Prometheus

- exposed on port `9090`
- configured with a read-only bind mount for the main configuration file
- configured with a read-only bind mount for alert rules
- attached to a persistent data volume

Grafana

- exposed on port `3001`
- attached to a persistent data volume for local configuration and dashboard persistence
- port remapped due to an existing service already occupying the default Grafana host port in the environment

Node Exporter

- exposed on port `9100`
- configured with host filesystem visibility through `/host`
- uses `pid: host` for improved host-level metric visibility

Restart Policy

The `restart: unless-stopped` policy was used to improve restart continuity after host reboot, provided the containers remain existing and are not explicitly removed.

8.2 File Verification

```
cat ~/observability/docker-compose.yml
```

9. Port Availability Verification

Before deploying the solution, port availability was verified:

```
ss -tulpen | grep -E '3001|9090|9100'
```

If no output is returned, the target ports are available.

10. Deployment Procedure

From the solution directory:

```
cd ~/observability
docker compose up -d
```

10.1 Purpose of the Command

- creates containers when missing
- starts the services in detached mode
- applies the Compose-defined configuration

10.2 Verify Service Status

```
docker compose ps
```

Expected state:

- `observability_grafana` -> `Up`
- `observability_prometheus` -> `Up`
- `observability_node_exporter` -> `Up`

11. Initial Validation Procedure

Once the containers were up, the first step was to make sure the stack was actually reachable before spending time inside Grafana.

Node Exporter was checked first:

```
curl http://localhost:9100/metrics | head
```

That returned the expected plain text metrics, which was a quick confirmation that the exporter was exposing host data correctly.

Prometheus health was then checked through its health endpoint:

```
curl http://localhost:9090/-/healthy
```

Expected response:

```
Prometheus is Healthy.
```

After that, the three local endpoints were opened in the browser to make sure there were no obvious connectivity or port mapping issues:

```
http://localhost:3001
http://localhost:9090
http://localhost:9100/metrics
```

12. Grafana Access and Initial Authentication

Grafana was accessed through:

```
http://localhost:3001
```

Initial credentials:

```
username: admin
password: admin
```

At first login, Grafana requires a password update.

13. Grafana-Prometheus Integration

Inside Grafana, the Prometheus data source was configured using the following procedure:

1. navigate to `Connections`
2. select `Add new data source`
3. choose `Prometheus`
4. configure the data source URL as:

```
http://prometheus:9090
```

1. select `Save & test`

13.1 Technical Rationale

The hostname `prometheus` was used instead of `localhost` because both Grafana and Prometheus run inside the same Docker network. From within the Grafana container, `localhost` would refer to Grafana itself rather than the Prometheus service.

14. Prometheus Target Verification

Prometheus targets were reviewed at:

```
http://localhost:9090/targets
```

The following targets were expected to appear in `UP` state:

- `prometheus`
- `node_exporter`

The following query was also executed:

```
up
```

Expected result:

- value `1` for Prometheus
- value `1` for Node Exporter

15. Dashboard Integration

To provide immediate host-level observability, a community dashboard was imported into Grafana.

15.1 Import Procedure

1. navigate to `Dashboards`
2. select `New`
3. select `Import`
4. enter dashboard ID:

```
1860
```

1. select `Load`
2. assign the Prometheus data source
3. select `Import`

15.2 Functional Outcome

The imported dashboard provided visibility into:

- CPU usage
- memory utilization
- disk utilization
- filesystem metrics
- uptime
- network traffic
- load averages

16. Functional Validation Using Controlled CPU Load

After the dashboard was connected, the next step was to check whether the graphs reacted to something real instead of just assuming the setup was correct.

The first test used a simple command:

```
yes > /dev/null
```

That did increase activity, but much less than expected at the dashboard level. The reason became obvious once the host CPU layout was taken into account: a single `yes` process was only saturating one logical core, so the overall percentage stayed relatively low.

To make the effect more visible, several background processes were started:

```
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
```

Running processes were checked with:

```
pgrep -a yes
```

And later stopped with:

```
pkill yes
```

This gave a much clearer signal in Grafana and also exposed a useful distinction between `CPU Busy` and `Sys Load`. CPU percentage moved upward, but system load often moved faster and looked more aggressive. In practice, that was a good reminder that utilization and scheduler pressure are related, but not identical, especially in a virtualized environment.

17. Alert Rule Definition

A rules directory was created:

```
mkdir -p ~/observability/prometheus/rules
```

Then the alert rule file was created:

```
nano ~/observability/prometheus/rules/alerts.yml
```

Final configuration:

```
groups:
  - name: node_alerts
    rules:
      - alert: HostHighCPU
        expr: 100 - (avg by(instance)
(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100) > 80
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "High CPU on {{ $labels.instance }}"
          description: "CPU utilization remained above 80% for 2 minutes."

      - alert: HostDiskAlmostFull
        expr: (100 - ((node_filesystem_avail_bytes{mountpoint="/",fstype!
~"tmpfs|overlay"} * 100) / node_filesystem_size_bytes{mountpoint="/",fstype!
~"tmpfs|overlay"})) > 85
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "Disk almost full on {{ $labels.instance }}"
          description: "The root filesystem exceeded 85% usage for 2
minutes."

      - alert: NodeExporterDown
        expr: up{job="node_exporter"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Node Exporter down on {{ $labels.instance }}"
          description: "Prometheus stopped scraping metrics from
node_exporter."
```

17.1 Alerting Scope

The alerting set was intentionally limited to a baseline operational scope in this implementation example:

- sustained CPU saturation
- critical disk utilization on the root filesystem
- exporter unavailability

In an enterprise production context, this would typically be expanded with notification routing, service-specific thresholds, environment-specific tuning, and ownership mapping.

18. Prometheus Rule Loading Configuration

The Prometheus main configuration was then updated to include alert rules.

Command:

```
nano ~/observability/prometheus/prometheus.yml
```

Final configuration:

```
global:
  scrape_interval: 15s

rule_files:
  - /etc/prometheus/rules/alerts.yml

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

18.1 Key Addition

```
rule_files:
  - /etc/prometheus/rules/alerts.yml
```

This instructs Prometheus to load and evaluate the rules defined in `alerts.yml`.

19. Applying Configuration Changes

Whenever configuration changes had to be applied, the following command was used:

```
cd ~/observability
docker compose up -d
```

When containers already existed and only operational stop/start control was needed, the following commands were used:

```
docker compose stop
docker compose start
```

19.1 Operational Note on `down`

`docker compose down` should not be used as a routine stop mechanism when restart continuity is desired, because it removes the stack containers. This has a direct effect on reboot behavior, since removed containers cannot be restarted automatically.

20. Alert Validation Procedure

The `HostHighCPU` rule was validated by generating CPU load again and watching the alert state in Prometheus:

```
http://localhost:9090/alerts
```

The expected behavior was simple: the alert should remain `inactive` under normal conditions, move to `pending` once the threshold was crossed, and finally transition to `firing` after the configured time window elapsed.

That sequence was observed during testing. After the background load was stopped with:

```
pkill yes
```

the alert returned to `inactive` after the evaluation window cleared.

What mattered here was not just the rule syntax. The test showed that the full path was working end to end: host metrics were being scraped, Prometheus was evaluating the expression, the threshold was reachable under controlled conditions, and the alert recovered normally once the pressure disappeared.

21. Restart Continuity and Host Startup Behavior

Docker was verified as enabled at host startup.

The containers were also created with the restart policy:

```
restart: unless-stopped
```

This allows the containers to restart automatically when Docker starts again, provided that:

- the containers still exist
- they were not explicitly removed using `docker compose down`

The restart policy was verified with:

```
docker inspect observability_grafana --format
'{{ .HostConfig.RestartPolicy.Name }}'
docker inspect observability_prometheus --format
'{{ .HostConfig.RestartPolicy.Name }}'
docker inspect observability_node_exporter --format
'{{ .HostConfig.RestartPolicy.Name }}'
```

Expected result:

```
unless-stopped
```

22. Operational Command Reference

Check stack status

```
cd ~/observability
docker compose ps
```

Start or recreate the stack

```
cd ~/observability
docker compose up -d
```

Stop containers without removing them

```
cd ~/observability
docker compose stop
```

Start existing containers

```
cd ~/observability
docker compose start
```

View full logs

```
cd ~/observability
docker compose logs
```

View logs by service

```
cd ~/observability
docker compose logs grafana
docker compose logs prometheus
docker compose logs node-exporter
```

Remove the stack

```
cd ~/observability
docker compose down
```

23. Findings and Implementation Notes

A few details during the implementation were more useful than they may look on paper.

The first one was port allocation. Grafana could not be exposed on the default host port because that port was already in use. Remapping it to `3001` was trivial from a technical standpoint, but it was still the right call because it avoided touching an existing service just to preserve a default.

The second point came from validation. At first glance, the dashboard seemed to underreport CPU activity when a single stress process was launched. In reality, the issue was not the dashboard at all; it was the test method. Once multiple processes were used, the behavior became much more representative. That small detour was useful because it forced a more careful reading of host metrics instead of a superficial one.

Another practical lesson was container lifecycle handling. In day-to-day work, `docker compose stop` and `docker compose down` can look interchangeable when someone just wants to “turn something off.” They are not. That difference became important when checking restart behavior after reboot, because removed containers obviously do not come back on their own.

24. Confidentiality and Publication Adaptation

This report has been adapted for publication as an implementation sample.

The following categories of information have been intentionally withheld or generalized:

- customer identity
- internal service names
- business application context
- infrastructure ownership references

- organization-specific operational routing
- environment-specific governance controls
- any identifiers that could expose the original implementation context

This adaptation preserves the implementation logic and technical reasoning while maintaining confidentiality.

25. Conclusion

This implementation covered an end-to-end observability flow:

- deployment of the stack
- integration of services
- host metrics collection
- dashboard-based visibility
- rule-based alerting
- controlled validation under load
- operational lifecycle handling
- restart continuity design

As a publication-adapted technical report, this document is intended to demonstrate implementation structure, operational discipline, and documentation quality suitable for SRE, DevOps, Platform Engineering, or technical consulting contexts.